Since 1983

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Erasure-Coded File Transfer over UDP

Recovering lost packets with GF(256) random linear coding

Mini-project presentation

Computer Networks Laboratory (CSE20110)
Department of Computer Engineering and Technology

Shreya Deshpande- 1262241054 Atharva Tawde- 1262240700
Tanvi Hardas-1262240292 Lohitaksh Bisen-1262243737

Group No. - 4 Panel K
Faculty: Ms. Uma Pujeri
Academic Year: 2026–2027

Problem and Objectives

Problem

UDP gives no delivery guarantee: packets can be lost, reordered or corrupted.

TCP repairs loss with ACKs and retransmissions, which add round-trip delay and need a return path for every loss.

Scope: client–server file transfer (up to about 2.8 MB) that survives packet loss using forward error correction.

Retransmission (TCP-style)



Lost packets are reported back, then sent again.

Needs a return path and waits one round trip per repair.

Cost: extra delay and feedback.

Objectives

1. Rebuild the file bit-for-bit (CRC-32 verified) at 30% or more packet loss, without per-packet retransmission.

2. Keep decoding overhead near zero: about K packets needed for K blocks.

3. Discard corrupted packets and run the same code on Windows and macOS.

Erasure coding (this project)



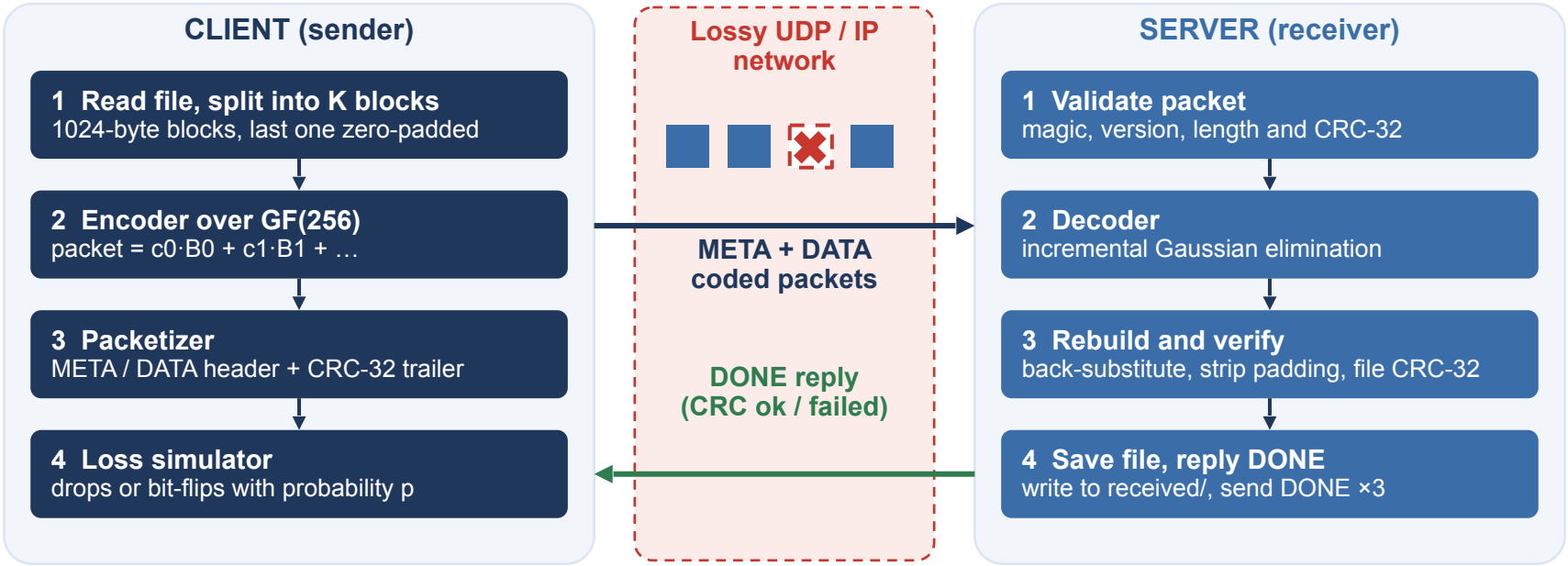
Extra coded packets are sent up front.

The spare packets cost bandwidth (about 1/(1−p) times the file), not time.

Any K independent packets rebuild the file.

Refs: [1] UDP (RFC 768), [2] random linear network coding.

Proposed Design and Tools



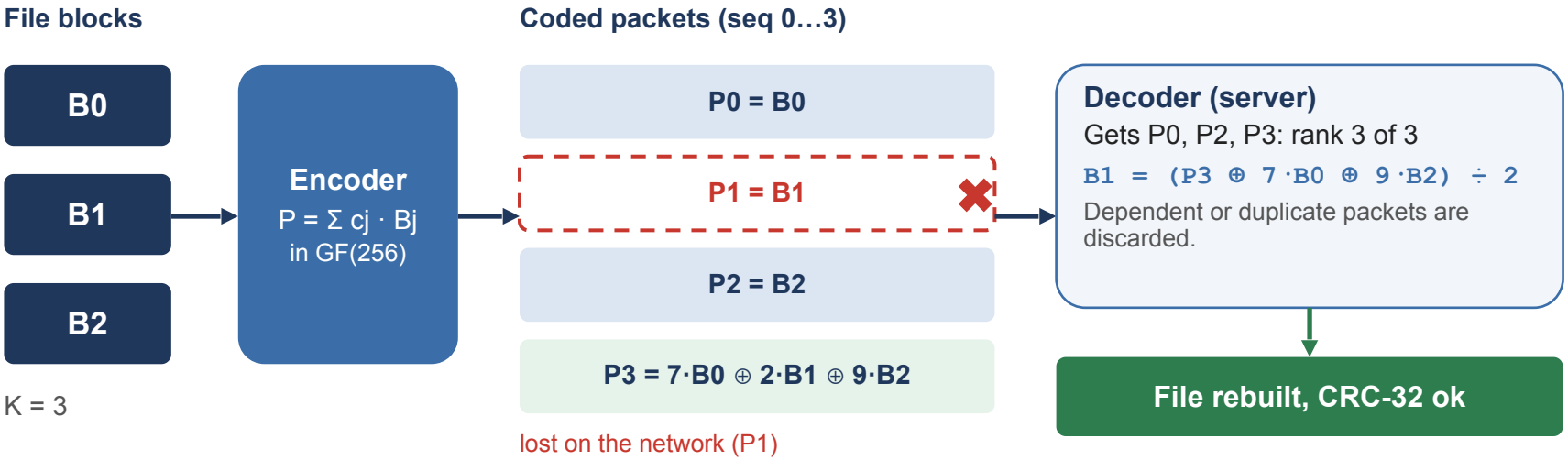
Tools: C++14 (g++ on Windows MinGW, Apple clang 21 on macOS), Winsock2 / BSD sockets, VS Code, Git; Python 3 + React visual lab.

Protocols: UDP for transport; own packet format (META, DATA, DONE) with magic number, version and CRC-32 on every packet.

Addressing: UDP port 9000, numeric IPv4 (127.0.0.1 or LAN IP); packet size at most 1472 B, so no IP fragmentation on a 1500 B MTU.

Refs: [1] UDP (RFC 768), [5] Winsock 2 documentation.

Implementation and Working



Coefficients are regenerated on both sides from (file_id, seq, K), so they are never sent. ⊕ is XOR (addition in GF(256)).

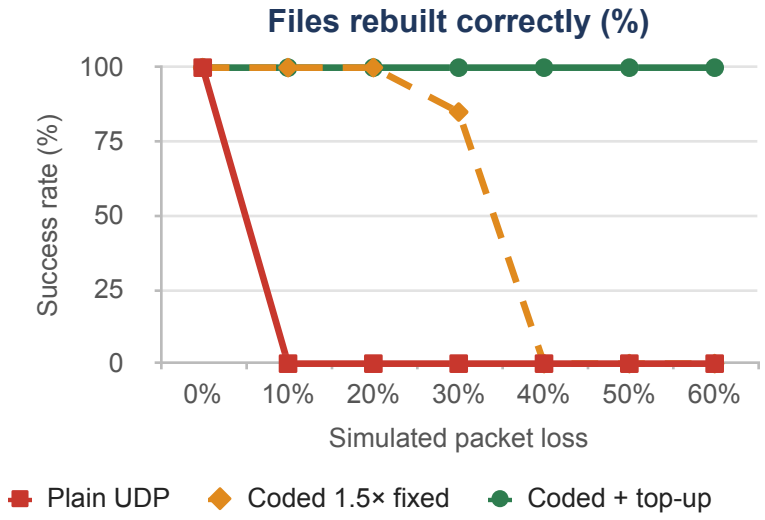
Implementation: net_compat.h (sockets), crc32.h, packet.h (wire format), coding.cpp (GF(256), encoder, decoder), client.cpp, server.cpp, sha256.h, trace.h (visual lab).

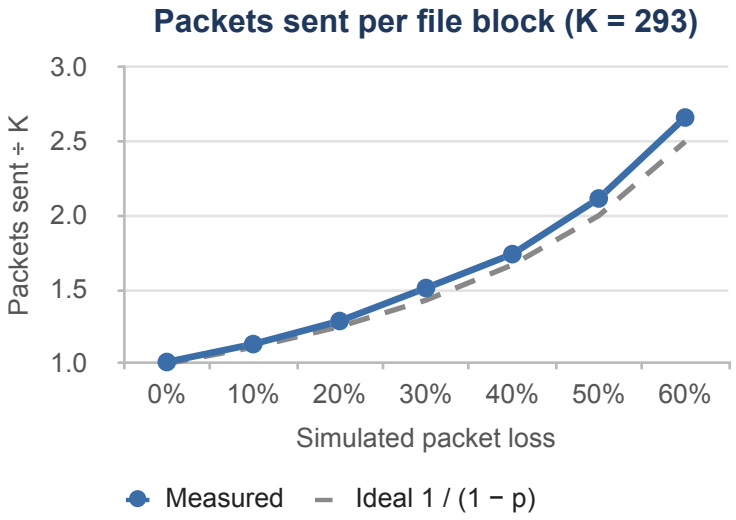
Working: client sends META, then packets 0…K−1 (plain blocks) and K… (random repair); server decodes as packets arrive and replies DONE; if DONE is missing, the client keeps sending fresh repair packets.

Troubleshooting: at 50% loss a 1 MB transfer took 19.1 s because top-up waited 0.3 s per round; a 50 ms pause growing to 250 ms cut it to 2.8 s (loopback test).

Refs: [2] Ho et al., random linear network coding; [4] Reed and Solomon; [6] Plank, GF arithmetic tutorial.

Testing and Results





Test: 300 KB file (K = 293), client to server on loopback, loss 0–60%, 20 runs per setting; plus 1 MB at 30% loss on the lab Mac.

Expected: plain UDP fails once any packet is lost; coded transfer succeeds while spare packets cover the loss.

Observed: plain UDP worked only at 0% loss; coded with a fixed 1.5× spare held to 20% loss (85% at 30%); coded + top-up rebuilt all 140 files, sending 1.02–1.07× the ideal 1/(1−p).

Objectives achieved: exact rebuild at 30% or more loss; decoder needed K packets (1 wasted in 217 coded runs); CRC rejected damaged packets.

Working Demonstration

Function to demonstrate

Send a 1 MB file over UDP with 30% simulated packet loss and rebuild it exactly at the server.

Steps

1. Start the server
   `./server 9000 received --once`
2. Send the file with 30% loss
   `./client 127.0.0.1 9000 tests/sample.bin --loss 0.3`
3. Compare sent and received files
   `cmp tests/sample.bin received/sample.bin`

```
$ ./client 127.0.0.1 9000 \
    tests/sample.bin --loss 0.3
K = 977 blocks of 1024 bytes
first batch: 1466 packets, loss 30%
dropped by simulator : 445 of 1480
time : 0.59 s
server reply : DONE, file rebuilt
  and CRC ok (decoder used 977
  packets for K = 977)
$ cmp tests/sample.bin \
    received/sample.bin
(no output = files identical)
```

Output recorded from the lab Mac run (macOS, Apple clang).
Live visual demo: python3 app.py.

Success criterion: client prints “DONE, file rebuilt and CRC ok”, the server used about K packets, and cmp prints nothing.

Conclusion and Contributions

1 Outcome

Working UDP client and server that rebuilt every test file exactly at up to 60% simulated loss, needing about K packets to decode. A web visual lab replays real runs step by step.

2 Application

Lossy or high-latency links (satellite, Wi-Fi, broadcast or one-way links) where retransmission is slow or impossible.

3 Limitation

K is limited to 2048 blocks (about 2.8 MB per file); decoding cost grows with K.

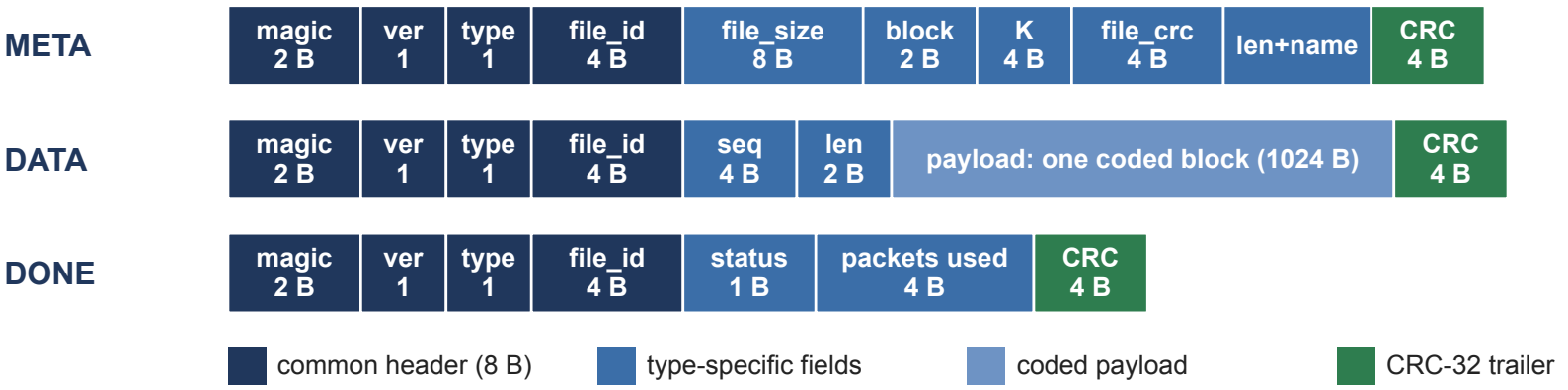
4 Future scope

Generation-based coding for large files, adaptive redundancy from server feedback, comparison with TCP.

Team contributions: [Name – actual responsibility] [Name – actual responsibility] [Name – actual responsibility] [Name – actual responsibility]

AI use: Claude (Anthropic) assisted with drafting the C++ code, tests and this presentation; every part was compiled, run and checked by the team (self-tests, CRC-verified transfers, cmp).

Backup: Packet Format



Byte order: all fields are written byte by byte in network (big-endian) order, so Windows x86 and macOS ARM agree. Field widths above are not to scale.

Header: magic 0xEC5A, version 1, type (1 META, 2 DATA, 3 DONE) and a random 32-bit file_id that identifies one transfer.

META: file size, block size, K = ceil(size ÷ block), CRC-32 of the whole file, file name; repeated so it survives loss.

DATA: sequence number plus one coded block. Coefficients are not sent: both sides derive them from (file_id, seq, K).

Validation: wrong magic, version, length, field value or CRC means the datagram is ignored. Largest packet is 1472 B (fits one Ethernet frame).

Backup: Coding and Decoding

Encoding (client)

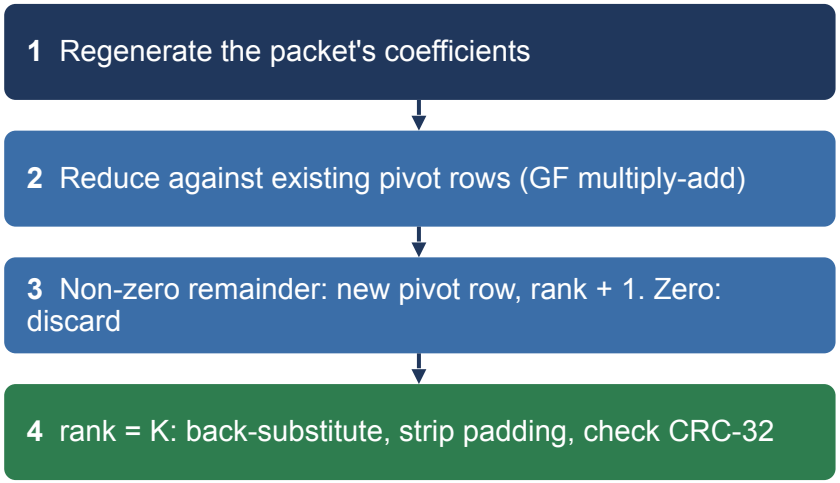
GF(256) with polynomial $x^8+x^4+x^3+x^2+1$ (0x11D); log/exp tables, addition is XOR.

Packet seq: Pseq = Σ cj · Bj over all K blocks.

Packets 0…K−1 use unit vectors (plain blocks, systematic); packets K and up use pseudo-random cj from a splitmix64 generator seeded by (file_id, seq).

Same seed on both sides, so the recipe never travels.

Decoding (server)



Why about K packets are enough

A random vector over GF(256) is independent of the rows already held with probability at least 1 − 1/256, so at most about one packet in 256 is wasted.

Packets that must be sent to get K through a link with loss p: about K ÷ (1 − p). At 30% loss that is 1.43 × the file size.

Decoding costs roughly $K^2 \cdot (K + \text{block size})$ byte operations, which is why K is capped at 2048.

Backup: Experiment Details

Loss	Plain UDP success	Coded 1.5× no top-up	Coded + top-up	Packets sent ÷ K	Ideal 1/(1−p)	Avg time (s)
0%	100%	100%	**100%**	1.01	1.00	0.10
10%	0%	100%	**100%**	1.13	1.11	0.11
20%	0%	100%	**100%**	1.29	1.25	0.13
30%	0%	85%	**100%**	1.51	1.43	0.15
40%	0%	0%	**100%**	1.74	1.67	0.17
50%	0%	0%	**100%**	2.12	2.00	0.22
60%	0%	0%	**100%**	2.66	2.50	0.27

Setup: 300,000-byte random file, K = 293 blocks of 1024 B, client and server as separate processes on loopback (127.0.0.1), 20 runs per cell, loss simulated in the client, files compared with cmp.

Plain UDP: only the K source packets are sent, no spare and no top-up.

Top-up: the client keeps sending fresh repair packets until the server answers DONE; time grows with loss because more packets must be sent.

References

[1] J. Postel, “User Datagram Protocol,” RFC 768, IETF, Aug. 1980.

[2] T. Ho, M. Médard, R. Koetter, D. R. Karger, M. Effros, J. Shi and B. Leong, “A random linear network coding approach to multicast,” IEEE Trans. Inf. Theory, vol. 52, no. 10, pp. 4413–4430, Oct. 2006.

[3] M. Luby et al., “RaptorQ forward error correction scheme for object delivery,” RFC 6330, IETF, Aug. 2011.

[4] I. S. Reed and G. Solomon, “Polynomial codes over certain finite fields,” J. SIAM, vol. 8, no. 2, pp. 300–304, 1960.

[5] Microsoft, “Windows Sockets 2 (Winsock) documentation,” learn.microsoft.com/windows/win32/winsock/.

[6] J. S. Plank, “A tutorial on Reed-Solomon coding for fault-tolerance in RAID-like systems,” Softw. Pract. Exper., vol. 27, no. 9, pp. 995–1012, 1997.

Project source code: github.com/shreyadeshpande0656/CN_miniproject